
REST: RESHAPING TOKEN-LEVEL POLICY GRADIENTS FOR TOOL-USE LARGE LANGUAGE MODELS

Zihan Lin[†]

CRIPAC, Institute of Automation
zihan.lin@nlpr.ia.ac.cn

Xiaohan Wang[†]

Meituan
wangxiaohan17@meituan.com

Jie Cao

CRIPAC, Institute of Automation
jie.cao@cripac.ia.ac.cn

Jiajun Chai

Meituan
chaijiajun@meituan.com

Guojun Yin^{*}

Meituan
yinguojun02@meituan.com

Wei Lin

Meituan
linwei31@meituan.com

Ran He

CRIPAC, Institute of Automation
rhe@nlpr.ia.ac.cn

[†] Equal contribution ^{*} Corresponding author

February 5, 2026

ABSTRACT

Large language models (LLMs) transcend passive generation and act as goal-directed agents by invoking external tools. Reinforcement learning (RL) offers a principled framework for optimizing these emergent tool-use policies, yet the prevailing paradigm relies exclusively on sparse outcome rewards and lacks consideration of the particularity of tool-use tasks, inflating policy-gradient variance and resulting in inefficient training. To better understand and address these challenges, we first establish a theoretical link between policy entropy and training stability of tool-use tasks, which reveals that structured, low-entropy tokens are primary determinants of rewards. Motivated by this insight, we propose **Reshaped Token-level policy gradients (ResT)** for tool-use tasks. ResT reshapes the policy gradient through entropy-informed token reweighting, progressively upweighting reasoning tokens as training proceeds. This entropy-aware scheme enables a smooth shift from structural correctness to semantic reasoning and stabilizes convergence in multi-turn tool-use tasks. Evaluation on BFCL and API-Bank shows that ResT achieves state-of-the-art results, outperforming prior methods by up to 8.76%. When fine-tuned on a 4B base LLM, ResT further surpasses GPT-4o by 4.11% on single-turn tasks and 1.50% on multi-turn base tasks.

1 Introduction

The recent emergence of large language models (LLMs) has precipitated a paradigm shift from passive, data-driven models to proactive, goal-oriented autonomous agents with complex tool-use abilities [1, 2]. Consequently, considerable research has focused on post-training LLMs to enhance their tool-use capabilities for domain-specific applications such as deep search and travel assistance. [3, 4]. Post-training for agentic LLMs centers on supervised fine-tuning (SFT) and Reinforcement learning (RL) with distinct trade-offs [5, 6]. In tool-use tasks, RL demonstrates strong generalization and robust performance [7–9]. Most approaches, however, assign a single outcome reward after multi-turn tool calls [10]; despite gains in multi-hop search [11], this paradigm introduces two challenges for general tool use: First, reward design is intrinsically noisy. Unlike multi-hop search with near-unique references, many practical tool-use tasks admit

multiple valid outputs (e.g., recommendations). As a result, outcome-only rewards induce high-variance gradients and provide weak incentives for reasoning, even when augmented with LLM-as-a-judge or learned reward models [12, 13]. Second, multi-turn tool-use tasks are systems-inefficient: per-sample interactions, concurrency control, and variable horizons depress throughput and inflate infrastructure cost, making tool-use RL substantially heavier than SFT and single-turn RL [14].

The challenges motivate a central question: Could a tool-use agent be trained by rewarding each turn with specific and low-variance rewards? Inspired by Deepseek-R1’s use of GRPO with rule-based rewards, we compute rule-based matching scores between generated responses and ground truth. Recent work, such as ToolRL [8], has validated this approach, achieving SOTA performance. Specifically, these studies decompose multi-turn tool calls into a series of single-turn tasks and establish rule-based matching rewards for efficient RL training. However, prior work often overlooks a crucial aspect: in tool-call settings, rule-based rewards are inherently concentrated on format tags, tool names, and key parameters, while reasoning and general-response tokens contribute little during the early stages of training. Uniformly treating all tokens dilutes the RL signal and is particularly detrimental for approaches lacking token-level critics (e.g., GRPO).

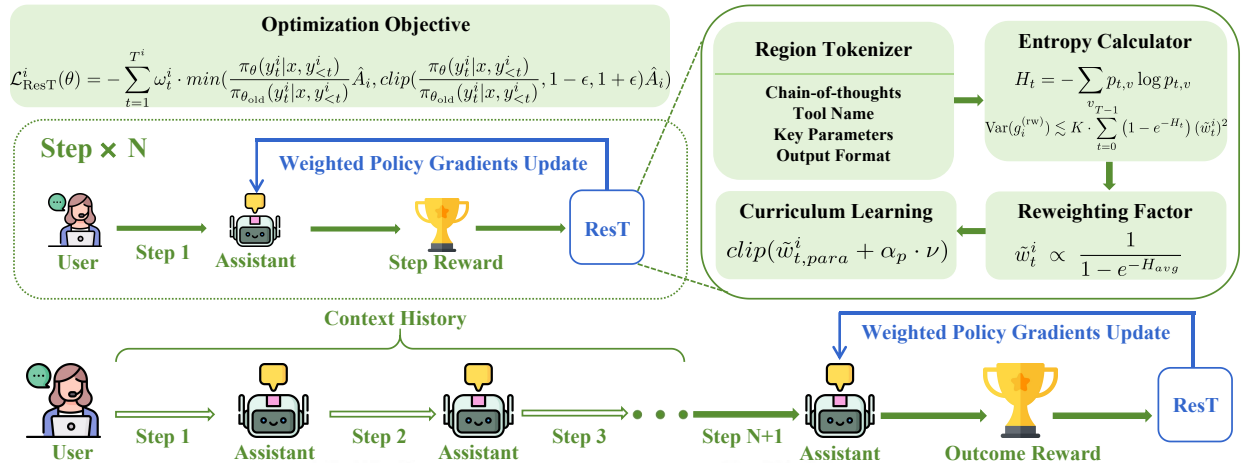


Figure 1: ResT decomposes multi-turn tool-use tasks into single-turn tasks and further reshapes the policy gradient according to the average entropy in different regions, enabling dense and effective reward signals.

To effectively and efficiently train tool-use agents, this paper proposes to **Reshape Token-level** policy gradients based on single-turn RL training (**ResT**). As introduced in Figure 1, ResT factorizes multi-turn tool use into single-turn steps and reshapes policy gradients with region-level average entropy. This design emphasizes structured, low-entropy tokens such as tool names and parameters, yielding dense and effective reward signals. While RL for tool-use agents should ideally progress from format compliance to parameter accuracy and ultimately complex reasoning, such curricula are seldom adopted. In ResT, this progression is guided by adaptive weighting across token regions, providing a lightweight curriculum that further stabilizes training and supports state-of-the-art results on tool-use benchmarks. The contributions are summarized as follows:

- 1) We establish a theoretical and empirical link between policy entropy and training stability. Specifically, we demonstrate that a lower average entropy correlates with reduced variance in policy-gradient updates. This insight reveals that structured, low-entropy tokens—such as tool names and parameters—are primary determinants of the resulting rewards. By uncovering this connection, we provide a principled explanation for why emphasizing these tokens during training leads to more stable optimization and improved performance on tool-use tasks.
- 2) We introduce an entropy-aware, token-level reshaping mechanism that incorporates curriculum learning. Within this framework, tool-use and reasoning tokens are weighted by their region-level average entropy, which directly influences training stability and regulates how different token categories contribute to reward signals. As training progresses and entropy decreases, the weights for reasoning tokens are gradually increased. This curriculum-based approach stabilizes convergence and systematically enhances the model’s reasoning capabilities in complex, multi-turn tool-calling scenarios.
- 3) Experimental results across multiple benchmark datasets show that our proposed method improves upon existing approaches by up to 8.76%, establishing a new state-of-the-art. Notably, when fine-tuned on the Qwen3-4B-2507 base model, our method outperforms GPT-4o [15] by 1.50% on multi-turn tool-use base tasks and 4.11% on single-turn

tool-use tasks. Furthermore, comprehensive ablation studies reveal that our curriculum-based reshaping outperforms static reward weighting by up to 4.86%.

2 Related Work

2.1 Reinforcement Learning for LLM Reasoning

Research on reinforcement learning (RL) for large language models (LLMs) progressed from early paradigms such as reinforcement learning from human feedback (RLHF) [16] and proximal policy optimization (PPO) [17] to a class of methods based on preferences and policies, including direct preference optimization (DPO) [18], SimPO [19], and group relative policy optimization (GRPO) [20]. Subsequent variants, such as dynamic sampling policy optimization (DAPO) [21] and value-augmented PPO (VAPO) [22], emphasize training stability and sample efficiency. Taken together, these advances indicate that RL can substantively enhance LLM reasoning, moving beyond surface-level pattern matching toward competencies such as logical inference and strategic planning [23, 24].

2.2 Tool-Use Agents

Tool use extends LLMs from passive text generators to interactive systems capable of invoking external tools and APIs [25, 26]. Existing approaches span prompt engineering and supervised fine-tuning (SFT) [27] on curated corpora [26, 28, 29], alongside specialized architectures and retrieval-augmented pipelines that select tools conditioned on the query [30]. Recent agentic frameworks further support multi-step planning and autonomous tool orchestration [14]. However, most methods remain predominantly supervised, limiting exploration of diverse reasoning strategies and generalization to novel tool-use scenarios [31].

To mitigate these limitations, RL formulations have been explored for tool-using agents. ToolRL [8] systematically compares SFT with GRPO [20]; GIGPO [9] adapts GRPO to multi-step tool use via multi-trajectory sampling and a two-level reward scheme; PVPO [32] further reduces rollout dependence and alleviates local optima through a reference–anchor mechanism and data pre-sampling; meanwhile, Agent Lightning [33] decouples training from execution and employs hierarchical RL to convert arbitrary agent trajectories into transition-level learning signals. Despite these advances, practical RL methods still largely optimize at the sequence level, where coarse-grained rewards induce high variance in policy gradients, constraining reward granularity and impeding efficient credit assignment and optimization.

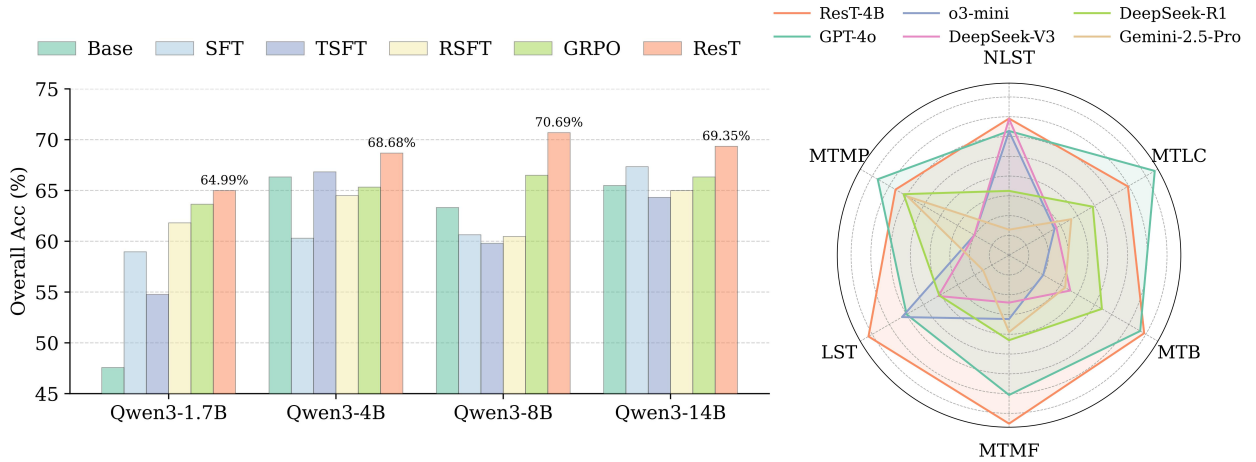


Figure 2: **Left:** Overall accuracy on the API-Bank test set. **Right:** Overall accuracy on the BFCL test set. Axes are: NLST: None-Live Single Turn, MTLC: Multi-Turn Long Context, MTB: Multi-Turn Base, MTMF: Multi-Turn w/ Missing Functions, LST: Live Single Turn, MTMP: Multi-Turn w/ Missing Parameters.

3 Method

In this section, we introduce two key innovations. First, we prove that lower average token entropy reduces policy-gradient variance and show that structured low-entropy tokens dominate the reward signal. Second, we propose token-level gradient reshaping, along with curriculum learning, to amplify updates for reward-critical tokens (e.g., tool names, arguments, output format), thereby mitigating variance and improving performance on complex tool-use tasks.

3.1 Theoretical Motivation

Successful tool-use hinges on generating a few critical tokens for names, parameters, and output formats. Conventional uniform rewards obscure these sparse signals, leading to high-variance gradients and unstable convergence. To address this, we introduce a theoretical framework for reducing policy gradient variance via entropy-aware token reweighting. Our analysis decomposes the gradient variance, links it to token-level entropy, and yields an optimal reweighting scheme that markedly improves training efficiency for high-precision tasks.

Lemma 1 (Policy Gradient Variance Decomposition). *Let $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$ denote the expected return. For a single trajectory $\tau_i = (y_{i,0:T-1})$, define the trajectory-level gradient*

$$g_i = \left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(y_{i,t} \mid x, y_{i,<t}) \right) \hat{A}_i, \quad (1)$$

where $\hat{A}_i$ is advantage function. Given G i.i.d. trajectories $\{\tau_i\}_{i=1}^G$ from π_θ , the mini-batch estimator is $\widehat{\nabla J} = (1/G) \sum_{i=1}^G g_i$. Then for each coordinate k the variance satisfies

$$\text{Var}(\widehat{\nabla J}^{(k)}) = \frac{1}{G} \text{Var}(g_i^{(k)}), \quad (2)$$

If $\hat{A}_i = (r_i - \mu)/\sigma$ with $\mu = \mathbb{E}[r]$ and $\sigma^2 = \text{Var}(r)$, then $\mathbb{E}[g_i] = \nabla_\theta J(\theta)$ and $\widehat{\nabla J}$ is unbiased. Proof is deferred to Appendix A.1.

Lemma 1 establishes the variance scaling law under mini-batch sampling. Building on this decomposition, we next characterize the second-order moment of g_i and connect it to token-level entropy.

Lemma 2 (Second-Order Moment and Entropy Connection). *For the single-trajectory estimator g_i , the variance satisfies*

$$\text{Var}(g_i) \leq \mathbb{E} \left[\hat{A}_i^2 \left\| \sum_{t=0}^{T-1} J_t^\top s_t \right\|^2 \right], \quad (3)$$

where $s_t = e_{y_t} - p_t$ is the logit-space score, $p_t = \text{softmax}(z_t)$ is the token distribution with z_t being the logits, and $J_t = \partial z_t / \partial \theta$ is the Jacobian. Moreover, the per-step score satisfies

$$\mathbb{E}[\|s_t\|^2] = 1 - \sum_v p_{t,v}^2 \leq 1 - e^{-H_t}, \quad (4)$$

where $H_t = -\sum_v p_{t,v} \log p_{t,v}$ is the Shannon entropy, with v indexing over the token vocabulary. Proof is deferred to Appendix A.2.

Lemma 2 shows that variance fundamentally depends on both squared advantage and token entropy. This result provides a natural handle for reweighting gradient contributions across timesteps. For a sequence of length T , we denote the reweighted policy gradient estimator as:

$$g_i^{(\text{rw})} := \left(\sum_{t=0}^{T-1} \tilde{w}_t \nabla_\theta \log \pi_\theta(y_{i,t} \mid x, y_{i,<t}) \right) \hat{A}_i, \quad \text{subject to} \quad \sum_{t=0}^{T-1} \tilde{w}_t = T, \quad (5)$$

Theorem 1 (Variance Upper Bound for Entropy-Aware Reweighting). *The variance of the reweighted estimator $g_i^{(\text{rw})}$ is upper-bounded using Lemma 2:*

$$\text{Var}(g_i^{(\text{rw})}) \leq \mathbb{E}[\hat{A}_i^2] \sum_{t=0}^{T-1} \beta_t (\tilde{w}_t)^2, \quad (6)$$

where $\beta_t = \mathbb{E}[\|J_t\|_F^2 (1 - e^{-H_t})]$ quantifies the variance contribution of step t by combining the Jacobian norm and the token entropy.

Building on this upper bound, we next seek the optimal reweighting scheme that minimizes the variance. This leads to a closed-form characterization of the weights.

Theorem 2 (Optimal Entropy-Aware Reweighting). *Minimizing the variance bound in Theorem 1 with respect to the weights $\tilde{w}_t$ yields a closed-form solution: (Proof is deferred to Appendix A.3.)*

$$\tilde{w}_t^* = \frac{T}{\sum_{u=0}^{T-1} \beta_u^{-1}} \cdot \frac{1}{\beta_t}, \quad (7)$$

which effectively down-weights sequence positions with larger intrinsic variance contributions. Substituting the optimal weights $\tilde{w}_t^*$ into Theorem 1 gives the minimized variance bound:

$$\text{Var}(g_i^{(\text{rw})}) \leq \mathbb{E}[\hat{A}_i^2] \cdot \frac{T^2}{\sum_{t=0}^{T-1} \beta_t^{-1}}. \quad (8)$$

In practice, computing β_t exactly may be computationally expensive. A practical alternative is to approximate β_t using entropy-only statistics, which gives simple weighting rules such as

$$\tilde{w}_t \propto \frac{1}{1 - e^{-H_{\text{avg}}}} \quad \text{or} \quad \tilde{w}_t \propto \frac{1}{H_{\text{avg}}}. \quad (9)$$

H_{avg} denotes the average entropy computed separately for each region (format tags, tool names, key parameters, and chain-of-thoughts) in tool-use tasks. These approximations preserve the inverse relationship between entropy and variance contribution, and thus serve as efficient surrogates for the optimal weights in Equation 7.

Our findings lead to a principled variance-reduction methodology for policy-gradient methods in tool-use tasks: open-ended tokens (e.g., chain-of-thought), characterized by higher entropy [34] (see Figure 4), are down-weighted to tighten variance bounds, improve sample efficiency, and preserve estimator unbiasedness.

3.2 Algorithm Design

3.2.1 Reward Score

Rule-based reward mechanisms have been empirically validated to achieve robust performance across diverse settings. In line with prior work [8, 10, 35, 36], our training objective incorporates components for both output format and execution correctness. Accordingly, we formulate the total reward as a weighted sum of a format matching score and a tool-calling correctness score. Format score of the i -th sample is determined by exact-match:

$$\mathcal{S}_{i,\text{format}} = \begin{cases} 1, & \text{if all required fields are complete in order} \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

The tool-calling correctness score is based on three components: tool name, parameter names, and parameter values. We measure name accuracy using the Jaccard similarity coefficient, defined as the intersection over union of the predicted P and ground-truth G sets:

$$r_{\text{name}} = \frac{|N_G \cap N_P|}{|N_G \cup N_P|} \in [0, 1], \quad r_{\text{para}} = \sum_{G_i \in G} \frac{|P_G \cap P_P|}{|P_G \cup P_P|} \in [0, |G|], \quad (11)$$

where N_G and N_P are the sets of ground-truth and predicted tool names, while P_G and P_P the corresponding sets of parameter names for a given tool. Due to the precision requirements of parameter values, we use an exact-match criterion for their evaluation:

$$r_{\text{value}} = \sum_{G_i \in G} \sum_{v \in \mathbf{v}(G_i)} \mathbf{1}[P_G[v] = P_P[v]] \in \left[0, \sum_{G_i \in G} |\mathbf{v}(G_i)|\right], \quad (12)$$

where $\mathbf{v}(\cdot)$ denotes the function that maps a graph G_i to its corresponding parameter value set. The normalized correctness score is then computed as:

$$\mathcal{S}_{i,\text{acc}} = \frac{r_{\text{name}} + r_{\text{para}} + r_{\text{value}}}{1 + |G| + \sum_{G_i \in G} |\mathbf{v}(G_i)|}. \quad (13)$$

The final dynamic scaling reward $\mathcal{R}_{\text{final}}$ is then calculated as a weighted sum:

$$\mathcal{R}_{i,\text{final}} = \beta_a \mathcal{S}_{i,\text{acc}} * (1 - \bar{v}) + \beta_f \mathcal{S}_{i,\text{format}} * (1 - \bar{v}), \quad (14)$$

where β denotes the relative importance and $\bar{v} \in (0, 1)$ denotes the training progress.

3.2.2 Reshaping Token-level Policy Gradients

ResT enables a more stable and effective use of rule-based rewards during model fine-tuning. Given a prompt containing user requirements and context, a policy π_θ generates G multi-turn response sequences composed of plain text and tool invocation segments. For the i -th sample in a group G , the advantage function $\hat{A}_i$ are then calculated:

$$\hat{A}_i = \frac{\mathcal{R}_{i,\text{final}} - \frac{1}{G} \sum_{i=0}^{G-1} \mathcal{R}_{i,\text{final}}}{\sqrt{\frac{1}{G} \sum_{i=0}^{G-1} (\mathcal{R}_{i,\text{final}} - \frac{1}{G} \sum_{i=0}^{G-1} \mathcal{R}_{i,\text{final}})^2 + \delta}}, \quad (15)$$

where $\delta > 0$ is a numerical stability constant.

We first tokenize the response and then partition it into four weight sets: $\tilde{w}_{t,fmt}$ (format tags), $\tilde{w}_{t,name}$ (tool names), $\tilde{w}_{t,para}$ (tool parameters), and $\tilde{w}_{t,thk}$ (chain-of-thought). Reweighting factor $\tilde{w}_t$ is initialized according to the entropy-based reweighting equation 9.

Our proposed curriculum learning method dynamically adjusts the reweighting factor based on the training progress ν , prioritizing different token types at different stages. Tool names consistently receive a high weight, reflecting their critical importance. Initially, the focus is on structural correctness via high format-token weights. Subsequently, the focus shifts to semantic accuracy by annealing format weights while increasing parameter-token weights:

$$\tilde{w}_{t,fmt}(\nu) = \max(w_{\min}, \tilde{w}_{t,fmt} - \alpha_f \cdot \nu), \quad \tilde{w}_{t,para}(\nu) = \min(w_{\max}, \tilde{w}_{t,para} + \alpha_p \cdot \nu), \quad (16)$$

where $[w_{\min}, w_{\max}]$ denotes the clipping interval and α denotes the magnitude of the update. To reduce policy-gradient variance, encourage step-by-step reasoning, and improve parameter accuracy, the weights for chain-of-thought and tool parameter tokens are synchronized:

$$\tilde{w}_{t,thk}(\nu) = \min(w_{\max}, \tilde{w}_{t,thk} + \alpha_t \cdot \nu). \quad (17)$$

We obtain the normalized reweighting factor by its mean over the sequence:

$$\hat{w}_t := \max(w_{\min}, \min(\tilde{w}_t, w_{\max})), \quad \bar{w} := \frac{1}{|T|} \sum_{t=1}^T \hat{w}_t, \quad w_t = \frac{\hat{w}_t}{\bar{w} + \delta}, \quad (18)$$

This normalization design enables the redistribution of learning signals among all tokens within the same sample, according to their relative policy entropy. By integrating the advantage function with the reweighting factor, we obtain the final optimization objective:

$$\mathcal{L}_{\text{ResT}}(\theta) = -\frac{1}{G} \sum_{i=0}^{G-1} \sum_{t=0}^{T-1} \frac{\omega_t}{T} \cdot \min(r_{i,t} \hat{A}_i, \text{clip}(r_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i), \quad (19)$$

where $r_{i,t} = \frac{\pi_{\theta}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}$ and ϵ limits the magnitude of gradient updates. The objective guides the policy toward structurally consistent and semantically accurate tool calls, while gradient reshaping alleviates variance in policy gradients, resulting in more stable and efficient alignment with task requirements. The complete algorithmic procedure is provided in Appendix B.

4 Experiments

4.1 Training Details

We curate a 4k mixed corpus to facilitate robust tool learning in RL: ToolACE [31] (2k samples) targets the invoke-vs-answer decision in multi-step interactions; Hammer (masked) [37] (1k samples) randomizes tool and argument names to enforce description-grounded generalization; and XLAM [29] (1k samples) supplies compositional tasks that require one or multiple tool calls per turn. Together, these sources supervise invocation timing, robust grounding, and compositional planning [8].

We convert multi-turn dialogue samples into single-step instances to provide denser and more fine-grained reward signals. Concretely, a dialogue with K interaction steps is decomposed into K sub-instances, each conditioned on the full preceding conversation as input and supervised with the model’s current action (e.g., tool invocation or response) as the target. This decomposition strategy, adopted from the SWiRL framework [38], substantially increases the density of training signals. Consequently, the model receives process-level supervision at each step, rather than only for the final outcome. This step-wise supervision fosters more stable learning of multi-step reasoning and tool use.

4.2 Main Results

We evaluate six baselines spanning supervised and reinforcement-learning regimes: **Base Model**: instruction-tuned without task-specific fine-tuning. **SFT**: supervised fine-tuning on the task training set. **TSFT**: SFT with up-weighted loss [39] on tool-call tokens to prioritize invocation accuracy. **RSFT**: SFT with up-weighted loss on reasoning tokens to strengthen intermediate reasoning. **GRPO**: RL-only method representative of the current state of the art for tool use. **SFT+GRPO**: initialize from an SFT checkpoint, then apply GRPO to isolate RL’s marginal gains.

Table 1: **BFCL multi-turn results (updated June 14, 2025)**. Metrics computed with official scripts. TSFT scales loss on tool-call tokens. RSFT scales loss on reasoning tokens. Best in **bold**.

Model	Overall Acc	Base	Miss Func	Miss Param	Long Context
Qwen3-1.7B (Base)	8.62%	12.00%	5.50%	9.50%	7.50%
Qwen3-1.7B (SFT)	14.25%	17.50%	14.50%	14.50%	10.50%
Qwen3-1.7B (TSFT)	13.50%	18.00%	14.50%	11.50%	10.00%
Qwen3-1.7B (RSFT)	15.00%	17.00%	16.50%	14.50%	12.00%
Qwen3-1.7B (GRPO)	14.50%	17.00%	15.50%	12.50%	13.00%
Qwen3-1.7B (SFT+GRPO)	14.88%	20.50%	18.00%	9.00%	12.00%
Qwen3-1.7B (Ours, ResT)	16.00%	20.00%	16.00%	15.00%	13.00%
Qwen3-4B-2507 (Base)	48.00%	57.00%	53.00%	33.50%	48.50%
Qwen3-4B-2507 (SFT)	43.00%	52.50%	46.50%	33.00%	40.00%
Qwen3-4B-2507 (TSFT)	48.25%	58.00%	52.00%	32.50%	50.50%
Qwen3-4B-2507 (RSFT)	41.25%	52.00%	47.50%	31.50%	34.00%
Qwen3-4B-2507 (GRPO)	41.62%	48.50%	45.50%	27.00%	45.50%
Qwen3-4B-2507 (SFT+GRPO)	40.38%	46.00%	42.00%	29.50%	44.00%
Qwen3-4B-2507 (Ours, ResT)	50.38%	62.50%	56.00%	33.00%	50.00%
Qwen3-8B (Base)	33.00%	38.50%	32.00%	31.50%	30.00%
Qwen3-8B (SFT)	28.75%	34.50%	33.50%	22.00%	25.00%
Qwen3-8B (TSFT)	29.00%	32.50%	34.00%	24.00%	25.50%
Qwen3-8B (RSFT)	25.12%	33.00%	29.50%	19.00%	19.00%
Qwen3-8B (GRPO)	36.00%	42.00%	42.00%	28.50%	31.50%
Qwen3-8B (SFT+GRPO)	32.38%	42.50%	35.00%	27.50%	24.50%
Qwen3-8B (Ours, ResT)	40.13%	50.50%	45.00%	32.00%	33.00%
Qwen3-14B (Base)	36.62%	48.00%	33.00%	33.50%	32.00%
Qwen3-14B (SFT)	28.12%	33.50%	30.00%	23.00%	26.00%
Qwen3-14B (TSFT)	27.62%	34.50%	34.00%	17.50%	24.50%
Qwen3-14B (RSFT)	27.00%	33.50%	27.50%	20.00%	27.00%
Qwen3-14B (GRPO)	38.88%	50.00%	43.50%	26.50%	35.50%
Qwen3-14B (SFT+GRPO)	34.12%	45.00%	37.50%	22.00%	32.00%
Qwen3-14B (Ours, ResT)	44.25%	57.00%	45.00%	36.50%	38.50%
GPT-4o-2024-11-20	50.00%	61.00%	45.50%	35.50%	58.00%
GPT-4.5-Preview-2025-02-27	45.38%	58.00%	37.50%	43.00%	43.00%
GPT-4.1-2025-04-14	40.50%	48.00%	34.00%	35.00%	45.00%
o3-mini-2025-01-31	28.75%	32.00%	26.00%	23.00%	34.00%
Grok-3-beta	45.50%	54.00%	39.00%	42.00%	47.00%
DeepSeek-V3-0324	29.87%	41.00%	21.00%	23.00%	34.50%
DeepSeek-R1-0528	38.88%	49.50%	31.50%	32.00%	42.50%
Gemini-2.5-Pro-Preview-05-06	34.62%	39.50%	29.50%	31.50%	38.00%

Benchmarks and Setup. The proposed method is rigorously evaluated on two comprehensive benchmarks: the Berkeley Function Calling Leaderboard (BFCL) [40], which assesses capabilities like multi-step tool use, and API-Bank [41], a three-tiered suite testing tool invocation in multi-turn dialogues. To isolate the algorithm’s contribution, several open-source models are fine-tuned against strong domain-specific baselines.

Performance Results. As reported in Figure 2, Table 1 and Table 2, fine-tuning the Qwen3 model family with ResT establishes a new state-of-the-art on both benchmarks. The method yields significant performance gains over the GRPO baseline, with improvements of up to 8.76% on BFCL and 3.02% on API-Bank under an identical data budget. Notably, the Qwen3-4B-2507 model fine-tuned with ResT even surpasses GPT-4o by 0.38% on multi-turn overall tool-use tasks, especially by 1.50% on multi-turn base tasks. On the single-turn tool-use tasks, it leads by 4.11%. These behaviors

Table 2: **API-Bank Test Results.** The results presented correspond to the highest score achieved by each method with its optimal hyperparameter settings. The evaluation dataset consists of 399 samples for Level 1, 67 for Level 2, and 131 for Level 3. Best in **bold**.

Model	Overall Acc	Level 1 Acc	Level 2 Acc	Level 3 Acc
Qwen3-1.7B (Base)	47.57%	53.38%	28.36%	39.69%
Qwen3-1.7B (SFT)	58.96%	66.92%	52.24%	38.17%
Qwen3-1.7B (TSFT)	54.77%	61.4%	43.28%	40.46%
Qwen3-1.7B (RSFT)	61.81%	67.92%	59.7%	44.27%
Qwen3-1.7B (GRPO)	63.65%	70.68%	61.19%	41.22%
Qwen3-1.7B (SFT+GRPO)	59.46%	65.91%	58.21%	40.46%
Qwen3-1.7B (Ours, ResT)	64.99%	70.18%	65.67%	48.85%
Qwen3-4B-2507 (Base)	66.33%	72.68%	64.18%	48.09%
Qwen3-4B-2507 (SFT)	60.30%	67.92%	59.70%	37.40%
Qwen3-4B-2507 (TSFT)	66.83%	72.93%	64.18%	49.62%
Qwen3-4B-2507 (RSFT)	64.49%	69.92%	59.7%	50.38%
Qwen3-4B-2507 (GRPO)	65.33%	71.93%	62.69%	46.56%
Qwen3-4B-2507 (SFT+GRPO)	68.51%	74.44%	65.67%	51.91%
Qwen3-4B-2507 (Ours, ResT)	68.68%	73.68%	64.18%	55.73%
Qwen3-8B (Base)	63.32%	70.68%	53.73%	45.80%
Qwen3-8B (SFT)	60.64%	69.17%	56.72%	36.64%
Qwen3-8B (TSFT)	59.80%	66.42%	50.75%	44.27%
Qwen3-8B (RSFT)	60.47%	67.42%	53.73%	42.75%
Qwen3-8B (GRPO)	66.50%	75.44%	64.18%	40.46%
Qwen3-8B (SFT+GRPO)	63.82%	72.18%	52.22%	42.75%
Qwen3-8B (Ours, ResT)	70.69%	74.94%	65.67%	60.31%
Qwen3-14B (Base)	65.49%	71.68%	56.72%	51.15%
Qwen3-14B (SFT)	67.34%	76.44%	62.69%	41.98%
Qwen3-14B (TSFT)	64.32%	68.67%	59.70%	53.44%
Qwen3-14B (RSFT)	64.99%	70.68%	55.22%	52.67%
Qwen3-14B (GRPO)	66.33%	73.18%	59.70%	48.85%
Qwen3-14B (SFT+GRPO)	67.34%	73.43%	58.21%	53.44%

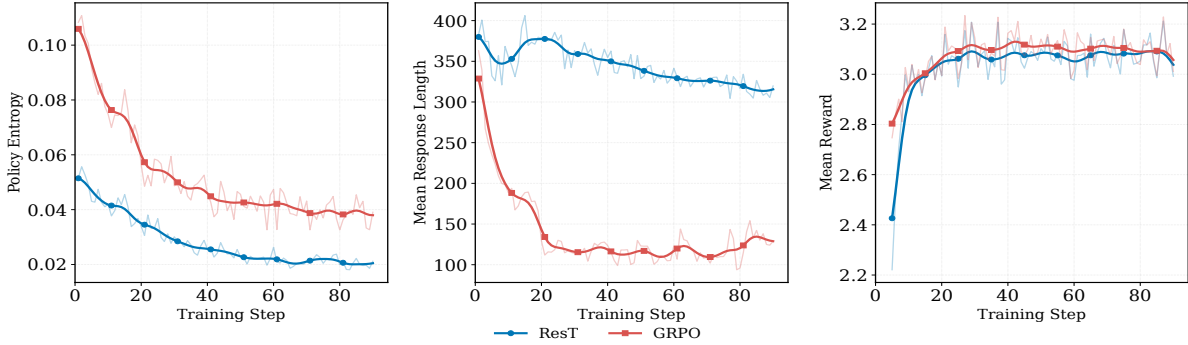


Figure 3: Learning curves for ResT and GRPO during training steps. The training dynamics show that ResT achieves a significantly lower and smoother policy entropy compared to GRPO, while maintaining comparable reward performance and longer responses.

indicate emergent forms of proactivity and metacognition, which enhance efficiency, mitigate hallucinations, and signal the emergence of foundational agentic intelligence.

Training Dynamics Analysis. The learning curves in Figure 3 highlight ResT’s advantage over GRPO. ResT’s policy entropy converges to ≈ 0.02 —lower and less volatile than GRPO’s ≈ 0.04 —indicating a more deterministic policy. This stability does not reduce performance: both reach similar final mean reward. Behaviorally, ResT sustains longer outputs (mean > 300 tokens), whereas GRPO collapses to ≈ 125 tokens. Overall, ResT stabilizes optimization and yields longer, more complex responses without sacrificing reward, avoiding the degradation observed with GRPO.

Table 3: **API-Bank Test Results for Relaxations.** No dynamic reward denotes no dynamic scaling reward value. No gradients for CoT denotes no gradient update for chain-of-thought. No curriculum learning assigns a constant weight to each segment. Best in **bold**.

Model	Overall Acc	Level 1 Acc	Level 2 Acc	Level 3 Acc
Qwen3-1.7B (<i>No dynamic reward</i>)	64.15%	69.92%	58.21%	49.62%
Qwen3-1.7B (<i>No gradients for CoT</i>)	62.98%	68.92%	62.69%	45.04%
Qwen3-1.7B (<i>No curriculum learning</i>)	61.14%	67.92%	58.21%	41.98%
Qwen3-1.7B (Ours, ResT)	64.99%	70.18%	65.67%	48.85%
Qwen3-4B-2507 (<i>No dynamic reward</i>)	64.15%	71.18%	64.18%	42.75%
Qwen3-4B-2507 (<i>No gradients for CoT</i>)	64.49%	71.18%	59.70%	46.56%
Qwen3-4B-2507 (<i>No curriculum learning</i>)	67.84%	73.18%	64.18%	53.44%
Qwen3-4B-2507 (Ours, ResT)	68.68%	73.68%	64.18%	55.73%
Qwen3-8B (<i>No dynamic reward</i>)	64.15%	71.68%	64.18%	41.22%
Qwen3-8B (<i>No gradients for CoT</i>)	66.33%	74.44%	65.67%	41.98%
Qwen3-8B (<i>No curriculum learning</i>)	65.83%	73.68%	62.69%	43.51%
Qwen3-8B (Ours, ResT)	70.69%	74.94%	65.67%	60.31%

4.3 Ablation Study

Ablations across base models (Table 3) show that each ResT component is necessary for peak performance. Disabling dynamic rewards, CoT policy gradients, or the curriculum consistently reduces accuracy. On Qwen3-8B, removing dynamic rewards lowers accuracy by 6.54%, and dropping CoT policy gradients and the curriculum yields declines of 4.36% and 4.86%, respectively. These results indicate complementary contributions and explain ResT’s robustness.

The ResT-trained model demonstrates robust generalization beyond its multi-turn training focus. As shown in Table 6, it performs comparably to the baseline on single-turn tasks and exhibits superior performance on real-time updated tasks, underscoring its out-of-distribution generalization. We also extend our evaluation to the Llama-3.2-3B-Instruct fine-tuned with ResT in Table 5 and Table 4 to verify that the effectiveness of our method is not confined to a specific model family. This confirms that the benefits of our approach are model-agnostic and highlights its broad applicability.

Table 4: **BFCL Results on Llama3.2-3B-Instruct model.** All metrics are calculated using the official scripts. Overall accuracies for the three different tasks are provided separately. Best in **bold**.

Model	Non-Live AST Acc	Live Acc	Multi Turn Acc	Relevance Detection	Irrelevance Detection
Llama-3.2-3B-Instruct (Base)	17.44%	43.85%	0.00%	77.78%	66.07%
Llama-3.2-3B-Instruct (SFT)	65.42%	63.04%	1.38%	77.78%	78.25%
Llama-3.2-3B-Instruct (TSFT)	57.06%	45.45%	0.12%	38.89%	59.17%
Llama-3.2-3B-Instruct (RSFT)	35.35%	60.20%	1.25%	94.44%	71.00%
Llama-3.2-3B-Instruct (GRPO)	74.38%	56.86%	1.37%	94.44%	62.23%
Llama-3.2-3B-Instruct (SFT+GRPO)	23.92%	45.40%	0.00%	11.11%	89.99%
Llama-3.2-3B-Instruct (Ours, ResT)	83.19%	61.53%	3.50%	88.89%	66.56%

5 Conclusion

We study tool-use in LLM agents through the lens of RL and establish a theoretical link between token-level policy entropy and training stability: lowering entropy tightens variance bounds on policy gradients, and reward mass concentrates on structured, low-entropy tokens such as tool names, arguments, and output-format markers. Building on this insight, we propose ResT, an entropy-aware, token-level RL algorithm that reshapes policy gradients via entropy-informed reweighting and a lightweight curriculum that progressively upweights reasoning tokens. This enables a smooth shift from structural correctness to semantic reasoning and stabilizes convergence in long-horizon, multi-turn tool use. Empirically, ResT attains state-of-the-art results on BFCL and API-Bank, improving prior methods by up to 8.76%; when fine-tuned on a 4B base LLM, it further surpasses GPT-4o by 4.11% on single-turn tasks and 1.50% on multi-turn base tasks. Future work will integrate verifiable structural rewards with judge-based semantic scores, learn schedules for entropy-aware mixing, and calibrate LLM-as-a-judge with uncertainty estimates.

References

- [1] Yuheng Cheng, Ceyao Zhang, Zhengwen Zhang, Xiangrui Meng, Sirui Hong, Wenhao Li, Zihao Wang, Zekai Wang, Feng Yin, Junhua Zhao, et al. Exploring large language model based intelligent agents: Definitions, methods, and prospects. *arXiv preprint arXiv:2401.03428*, 2024.
- [2] Yunjia Xi, Jianghao Lin, Yongzhao Xiao, Zheli Zhou, Rong Shan, Te Gao, Jiachen Zhu, Weiwen Liu, Yong Yu, and Weinan Zhang. A survey of llm-based deep search agents: Paradigm, optimization, evaluation, and challenges. *arXiv preprint arXiv:2508.05668*, 2025.
- [3] Hongru Wang, Cheng Qian, Manling Li, Jiahao Qiu, Boyang Xue, Mengdi Wang, Heng Ji, and Kam-Fai Wong. Toward a theory of agents as tool-use decision-makers. *arXiv preprint arXiv:2506.00886*, 2025.
- [4] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjuan Zhong. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.
- [5] Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- [6] Luping Wang, Sheng Chen, Linnan Jiang, Shu Pan, Runze Cai, Sen Yang, and Fei Yang. Parameter-efficient fine-tuning in large language models: a survey of methodologies. *Artificial Intelligence Review*, 58(8):227, 2025.
- [7] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Jirong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- [8] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- [9] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.
- [10] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- [11] Boyuan Liu, Feng Ji, Jiayan Nan, Han Zhao, Weiling Chen, Shihao Xu, and Xing Zhou. Omne-r1: Learning to reason with memory for multi-hop question answering. *arXiv preprint arXiv:2508.17330*, 2025.
- [12] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025.
- [13] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on llm-as-a-judge, 2025. URL <https://arxiv.org/abs/2411.15594>.
- [14] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [15] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [16] Timo Kaufmann, Paul Weng, Viktor Bengs, and Eyke Hüllermeier. A survey of reinforcement learning from human feedback. 2024.
- [17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [18] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- [19] Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward. *Advances in Neural Information Processing Systems*, 37:124198–124235, 2024.
- [20] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.

- [21] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- [22] Yu Yue, Yufeng Yuan, Qiying Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, Tiantian Fan, Zhengyin Du, et al. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.
- [23] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [24] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye, Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Weixin Xu, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, Zonghan Yang, and Zongyu Lin. Kimi k1.5: Scaling reinforcement learning with llms, 2025. URL <https://arxiv.org/abs/2501.12599>.
- [25] Yixian Zhang, Huaze Tang, Chao Wang, and Wenbo Ding. Policy newton algorithm in reproducing kernel hilbert space. *arXiv preprint arXiv:2506.01597*, 2025.
- [26] Bingguang Hao, Maolin Wang, Zengzhuang Xu, Cunyin Peng, Yicheng Chen, Xiangyu Zhao, Jinjie Gu, and Chenyi Zhuang. Funreason: Enhancing large language models’ function calling via self-refinement multiscale loss and automated data refinement, 2025. URL <https://arxiv.org/abs/2505.20192>.
- [27] Nuo Chen, Hongguang Li, Baoyuan Wang, and Jia Li. From good to great: Improving math reasoning with tool-augmented interleaved prompting, 2023. URL <https://arxiv.org/abs/2401.05384>.
- [28] Shengqian Qin, Yakun Zhu, Linjie Mu, Shaoting Zhang, and Xiaofan Zhang. Meta-tool: Unleash open-world function calling capabilities of general-purpose large language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the*

- Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30653–30677, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1481. URL <https://aclanthology.org/2025.acl-long.1481/>.
- [29] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Awalgaonkar, Rithesh Murthy, Eric Hu, Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. xlam: A family of large action models to empower ai agent systems, 2024. URL <https://arxiv.org/abs/2409.03215>.
 - [30] Xuan-Phi Nguyen, Shrey Pandit, Senthil Purushwalkam, Austin Xu, Hailin Chen, Yifei Ming, Zixuan Ke, Silvio Savarese, Caiming Xiong, and Shafiq Joty. Sfr-rag: Towards contextually faithful llms, 2024. URL <https://arxiv.org/abs/2409.09916>.
 - [31] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong Wang, Yuxian Wang, Wu Ning, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhi Wang, Yong Liu, Yasheng Wang, Duyu Tang, Dandan Tu, Lifeng Shang, Xin Jiang, Ruiming Tang, Defu Lian, Qun Liu, and Enhong Chen. Toolace: Winning the points of llm function calling, 2025. URL <https://arxiv.org/abs/2409.00920>.
 - [32] Wenfeng Feng, Penghong Zhao, Guochao Jiang, Chuzhan Hao, Yuewei Zhang, and Hao Wang. Pvp0: Pre-estimated value-based policy optimization for agentic reasoning. *arXiv preprint arXiv:2508.21104*, 2025.
 - [33] Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent lightning: Train any ai agents with reinforcement learning, 2025. URL <https://arxiv.org/abs/2508.03680>.
 - [34] Maciej Besta, Julia Barth, Eric Schreiber, Ales Kubicek, Afonso Catarino, Robert Gerstenberger, Piotr Nyczyk, Patrick Iff, Yueling Li, Sam Houlston, Tomasz Sternal, Marcin Copik, Grzegorz Kwaśniewski, Jürgen Müller, Łukasz Flis, Hannes Eberhard, Zixuan Chen, Hubert Niewiadomski, and Torsten Hoeffler. Reasoning language models: A blueprint, 2025. URL <https://arxiv.org/abs/2501.11223>.
 - [35] Minpeng Liao, Wei Luo, Chengxi Li, Jing Wu, and Kai Fan. Mario: Math reasoning with code interpreter output – a reproducible pipeline, 2024. URL <https://arxiv.org/abs/2401.08190>.
 - [36] Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning, 2025. URL <https://arxiv.org/abs/2502.14768>.
 - [37] Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, Xiaoyun Mo, Jiamu Zhou, Cheng Cheng, Yin Zhao, Jun Wang, and Weinan Zhang. Robust function-calling for on-device language model via function masking. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=yVQcr4qjD6>.
 - [38] Anna Goldie, Azalia Mirhoseini, Hao Zhou, Irene Cai, and Christopher D. Manning. Synthetic data generation & multi-step rl for reasoning & tool use, 2025. URL <https://arxiv.org/abs/2504.04736>.
 - [39] Mathew Huerta-Enochian and Seung Yong Ko. Instruction fine-tuning: Does prompt loss matter? In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22771–22795, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.1267. URL <https://aclanthology.org/2024.emnlp-main.1267/>.
 - [40] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023. URL <https://arxiv.org/abs/2305.15334>.
 - [41] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms, 2023. URL <https://arxiv.org/abs/2304.08244>.

A Proofs

This appendix provides detailed proofs of the theoretical results presented in Section 3.1.

A.1 Proof of Lemma 1 (Policy Gradient Variance Decomposition)

Proof. Setup. Let a trajectory be $\tau = (x, y_{0:T-1})$ sampled from π_θ , with return $R(\tau)$. For trajectory index i , write τ_i , $r_i := R(\tau_i)$, and $y_{i,t}$. The per-time-step score is $\nabla_\theta \log \pi_\theta(y_{i,t} \mid x, y_{i,<t})$.

Step 1. Policy gradient with a baseline. By the policy gradient theorem,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) R(\tau) \right].$$

Let $b_t = b_t(x, y_{<t})$ be any baseline that does not depend on the sampled action y_t given the state/history. Then, conditioning on $(x, y_{<t})$,

$$\begin{aligned} \mathbb{E}_{y_t \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) b_t] &= b_t \sum_{y_t} \pi_\theta(y_t \mid x, y_{<t}) \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \\ &= b_t \nabla_\theta \sum_{y_t} \pi_\theta(y_t \mid x, y_{<t}) = 0. \end{aligned}$$

Hence one may replace $R(\tau)$ by the advantage $A(\tau) := R(\tau) - \sum_t b_t$ inside the expectation without changing its value. In what follows we set $b_t \equiv 0$ for simplicity (equivalently, redefine μ, σ as the population moments of $A(\tau)$ if a baseline is used).

Step 2. Population standardization and unbiased direction. Consider the population-standardized advantage

$$\hat{A}_i = \frac{r_i - \mu}{\sigma}, \quad \mu := \mathbb{E}[r], \quad \sigma^2 := \text{Var}(r),$$

where expectations are under the current policy π_θ . Define the trajectory-level gradient

$$g_i = \hat{A}_i \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(y_{i,t} \mid x, y_{i,<t}).$$

Taking expectations and using $\mathbb{E}[r_i - \mu] = 0$,

$$\mathbb{E}[g_i] = \frac{1}{\sigma} \mathbb{E} \left[(r_i - \mu) \sum_t \nabla_\theta \log \pi_\theta(y_{i,t} \mid x, y_{i,<t}) \right] = \frac{1}{\sigma} \nabla_\theta J(\theta).$$

Thus g_i is an unbiased *direction* estimator for $\nabla_\theta J(\theta)$, up to a scalar factor $1/\sigma$. If instead one uses the centered (unnormalized) advantage $\hat{A}_i = r_i - \mu$, then $\mathbb{E}[g_i] = \nabla_\theta J(\theta)$ exactly. In practice the factor $1/\sigma$ can be absorbed into the learning rate.

Step 3. Mini-batch variance decomposition. Let $g_1, \dots, g_G$ be i.i.d. copies of g_i with finite second moments, and define the mini-batch estimator

$$\bar{g} = \frac{1}{G} \sum_{i=1}^G g_i.$$

Its covariance is

$$\text{Cov}(\bar{g}) = \mathbb{E}[(\bar{g} - \mathbb{E}[g_1])(\bar{g} - \mathbb{E}[g_1])^\top] = \frac{1}{G^2} \sum_{i=1}^G \sum_{j=1}^G \text{Cov}(g_i, g_j).$$

Independence gives $\text{Cov}(g_i, g_j) = 0$ for $i \neq j$, hence

$$\text{Cov}(\bar{g}) = \frac{1}{G^2} \sum_{i=1}^G \text{Cov}(g_i, g_i) = \frac{1}{G} \text{Cov}(g_1).$$

Taking the k -th diagonal entry yields, for any coordinate k ,

$$\text{Var}(\bar{g}^{(k)}) = \frac{1}{G} \text{Var}(g_1^{(k)}),$$

proving equation 2.

Step 4. Consistency checks. All symbols are trajectory-indexed (i) or time-indexed (t) but not conflated. The expectation and variance are with respect to the data-generating policy π_θ . Population moments (μ, σ) refer to returns under π_θ at the current parameters. The i.i.d. assumption holds when trajectories are drawn independently from π_θ . $\square$

Remark (batch normalization of advantages). If one instead uses the *batch-normalized* statistic

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G}, \quad \mu_G := \frac{1}{G} \sum_{j=1}^G r_j, \quad \sigma_G^2 := \frac{1}{G} \sum_{j=1}^G (r_j - \mu_G)^2,$$

then $\{g_i\}$ are no longer independent because each g_i is a function of the full batch. A direct expansion gives the exact decomposition

$$\text{Cov}(\bar{g}) = \frac{1}{G} \text{Var}(g_1) + \frac{G-1}{G} \text{Cov}(g_1, g_2),$$

where $\text{Cov}(g_1, g_2)$ denotes the cross-covariance induced solely by the shared batch statistics. Under mild moment and smoothness conditions, a delta-method (influence function) expansion of (μ_G, σ_G) around (μ, σ) yields $\text{Cov}(g_1, g_2) = O(1/G)$, so

$$\text{Cov}(\bar{g}) = \frac{1}{G} \text{Var}(g_1) + O\left(\frac{1}{G}\right),$$

and the bias of $\mathbb{E}[\bar{g}]$ relative to $\nabla_\theta J(\theta)/\sigma$ is $O(1/G)$. Both corrections vanish as $G \rightarrow \infty$, but for small G they can be non-negligible in practice.

A.2 Proof of Lemma 2 (Second-Order Moment–Entropy Connection)

Proof. Setup and notation. Fix a single trajectory. Let $z_t \in \mathbb{R}^V$ denote the logits, $p_t = \text{softmax}(z_t) \in \Delta^{V-1}$ the token distribution, and e_{y_t} the one-hot vector of $y_t \sim p_t$. Define the score vector $s_t := e_{y_t} - p_t \in \mathbb{R}^V$ and the Jacobian $J_t := \partial z_t / \partial \theta \in \mathbb{R}^{V \times d}$. Then

$$\nabla_{\theta} \log \pi_{\theta}(y_t \mid x, y_{<t}) = J_t^{\top} s_t, \quad g_i = \hat{A}_i \sum_{t=0}^{T-1} J_t^{\top} s_t.$$

All expectations below are taken over the joint randomness of the trajectory and the policy unless explicitly conditioned.

Step 1. Second-order moment control. By the definition of vector variance,

$$\text{Var}(g_i) = \mathbb{E}[\|g_i\|^2] - \|\mathbb{E}[g_i]\|^2 \leq \mathbb{E}[\|g_i\|^2] = \mathbb{E}\left[\hat{A}_i^2 \left\| \sum_{t=0}^{T-1} J_t^{\top} s_t \right\|^2\right]. \quad (*)$$

Condition on the trajectory prefix up to time t . Since

$$\mathbb{E}[s_t \mid x, y_{<t}] = 0,$$

the sequence $\{J_t^{\top} s_t\}_{t=0}^{T-1}$ forms a vector-valued martingale difference, and thus is L^2 -orthogonal. Therefore

$$\mathbb{E}\left[\left\| \sum_{t=0}^{T-1} J_t^{\top} s_t \right\|^2\right] = \sum_{t=0}^{T-1} \mathbb{E}[\|J_t^{\top} s_t\|^2].$$

Applying the inequality $\|J_t^{\top} s_t\| \leq \|J_t\|_F \|s_t\|$ gives

$$\mathbb{E}\left[\hat{A}_i^2 \left\| \sum_{t=0}^{T-1} J_t^{\top} s_t \right\|^2\right] \leq \mathbb{E}[\hat{A}_i^2] \sum_{t=0}^{T-1} \mathbb{E}[\|J_t\|_F^2 \|s_t\|^2] = \mathbb{E}[\hat{A}_i^2] \sum_{t=0}^{T-1} \beta_t \mathbb{E}[\|s_t\|^2],$$

where we set

$$\beta_t := \mathbb{E}[\|J_t\|_F^2] \quad \text{or more tightly} \quad \beta_t := \mathbb{E}[\|J_t\|_F^2 (1 - e^{-H_t})] \geq 0,$$

depending on whether we insert the entropy factor at this stage. Either choice preserves the quadratic-in-weights structure used later; we adopt the tighter definition when convenient.

Step 2. Exact expression for the expected score norm. Condition on p_t . Since $s_t = e_{y_t} - p_t$ with $y_t \sim p_t$,

$$\|s_t\|^2 = \|e_{y_t}\|^2 - 2\langle e_{y_t}, p_t \rangle + \|p_t\|^2 = 1 - 2p_{t,y_t} + \sum_{v=1}^V p_{t,v}^2.$$

Taking expectation over $y_t \sim p_t$ removes the index:

$$\mathbb{E}_{y_t \sim p_t}[\|s_t\|^2 \mid p_t] = 1 - \sum_{v=1}^V p_{t,v}^2 = 1 - \|p_t\|_2^2. \quad (\dagger)$$

Step 3. Entropy connection via the ordering $H_2 \leq H$. Since the Rényi-2 entropy $H_2(p_t) = -\log \sum_v p_{t,v}^2$ is always upper-bounded by the Shannon entropy H_t , we obtain

$$H_t = \mathbb{E}_{v \sim p_t}[-\log p_{t,v}] \geq -\log \mathbb{E}_{v \sim p_t}[p_{t,v}] = -\log\left(\sum_{v=1}^V p_{t,v}^2\right).$$

Equivalently,

$$\sum_{v=1}^V p_{t,v}^2 \geq e^{-H_t}. \quad (\ddagger)$$

Combining equation $\dagger$ and equation $\ddagger$ gives the entropy upper bound

$$\mathbb{E}[\|s_t\|^2 \mid p_t] = 1 - \|p_t\|_2^2 \leq 1 - e^{-H_t}.$$

Taking expectation over the randomness in p_t yields

$$\mathbb{E}[\|s_t\|^2] \leq 1 - \mathbb{E}[e^{-H_t}] \leq 1 - e^{-\mathbb{E}[H_t]},$$

where the last inequality uses convexity of $\exp(\cdot)$ (optional if one keeps $1 - \mathbb{E}[e^{-H_t}]$). Equality in equation ‡ holds iff p_t is uniform on its support, certifying tightness.

Step 4. Conclusion. Substituting the entropy bound for $\mathbb{E}[\|s_t\|^2]$ into equation * gives

$$\text{Var}(g_i) \leq \mathbb{E}[\hat{A}_i^2] \sum_{t=0}^{T-1} \mathbb{E}[\|J_t\|_F^2 \|s_t\|^2] \leq \mathbb{E}[\hat{A}_i^2] \sum_{t=0}^{T-1} \mathbb{E}[\|J_t\|_F^2 (1 - e^{-H_t})],$$

which is precisely the claimed second-order moment–entropy relation.

Remarks on assumptions and tightness. (i) The step factoring $\mathbb{E}[\hat{A}_i^2]$ is exact if $\hat{A}_i$ is independent of (J_t, s_t) or after conditioning on the trajectory prefix; without independence one may apply Hölder to absorb the mismatch into a constant, leaving the functional dependence on entropy unchanged. (ii) The bound $\mathbb{E}[\|s_t\|^2] = 1 - \|p_t\|_2^2 \leq 1 - e^{-H_t}$ becomes tight when p_t is uniform over its support, reflecting the ordering $H_2 \leq H$ between Rényi-2 and Shannon entropies. $\square$

A.3 Proof of Theorem 2 (Optimal Entropy-Aware Reweighting)

Proof. Step 1. Starting point. By Lemma 2 and the inequality $\|J_t^\top s_t\|^2 \leq \|J_t\|_F^2 \|s_t\|^2$,

$$\text{Var}(g_i) \leq \mathbb{E}\left[\hat{A}_i^2 \sum_{t=0}^{T-1} \|J_t\|_F^2 \|s_t\|^2\right].$$

Applying the entropy bound $\mathbb{E}[\|s_t\|^2] \leq 1 - e^{-H_t}$ gives

$$\text{Var}(g_i) \leq \mathbb{E}[\hat{A}_i^2] \sum_{t=0}^{T-1} \mathbb{E}[\|J_t\|_F^2 (1 - e^{-H_t})] = \mathbb{E}[\hat{A}_i^2] \sum_{t=0}^{T-1} \beta_t,$$

where we set

$$\beta_t := \mathbb{E}[\|J_t\|_F^2 (1 - e^{-H_t})] \geq 0.$$

Remark. Factoring out $\mathbb{E}[\hat{A}_i^2]$ tacitly assumes that $\hat{A}_i$ is independent of (J_t, H_t) or that the bound is taken conditionally on the trajectory; without independence one may invoke Hölder/Cauchy–Schwarz to obtain an equivalent quadratic objective in the weights, which preserves the optimal proportionality derived below.

Step 2. Effect of reweighting. Introduce per-step nonnegative weights $\tilde{w}_t \geq 0$ with normalization $\sum_{t=0}^{T-1} \tilde{w}_t = T$, and consider the reweighted estimator

$$g_i^{(\text{rw})} = \hat{A}_i \sum_{t=0}^{T-1} \tilde{w}_t J_t^\top s_t.$$

Variance scales quadratically in the weights, hence

$$\text{Var}(g_i^{(\text{rw})}) \leq \mathbb{E}[\hat{A}_i^2] \sum_{t=0}^{T-1} \beta_t (\tilde{w}_t)^2.$$

Step 3. Convex program. We minimize the separable quadratic

$$\min_{\{\tilde{w}_t\}} \sum_{t=0}^{T-1} \beta_t (\tilde{w}_t)^2 \quad \text{s.t.} \quad \sum_{t=0}^{T-1} \tilde{w}_t = T, \quad \tilde{w}_t \geq 0.$$

If $\beta_t > 0$ for all t , the objective is strictly convex and the feasible set is a simplex, so the minimizer is unique.

Step 4. Lagrangian and stationarity. We first solve the equality-constrained problem (the inequality will be verified inactive). The Lagrangian reads

$$\mathcal{L}(\tilde{w}, \lambda) = \sum_{t=0}^{T-1} \beta_t (\tilde{w}_t)^2 + \lambda \left(\sum_{t=0}^{T-1} \tilde{w}_t - T \right).$$

Taking the derivative with respect to $\tilde{w}_t$ gives the stationarity condition

$$\frac{\partial \mathcal{L}}{\partial \tilde{w}_t} = 2\beta_t \tilde{w}_t + \lambda = 0 \quad \implies \quad \tilde{w}_t = -\frac{\lambda}{2\beta_t} = \frac{c}{\beta_t} \quad \text{with } c := -\lambda/2.$$

Step 5. Normalization and interiority. Enforcing $\sum_{t=0}^{T-1} \tilde{w}_t = T$ yields

$$c \sum_{t=0}^{T-1} \beta_t^{-1} = T \quad \implies \quad c = \frac{T}{\sum_{u=0}^{T-1} \beta_u^{-1}}, \quad \lambda^* = -\frac{2T}{\sum_{u=0}^{T-1} \beta_u^{-1}} < 0.$$

Hence, for all t ,

$$\tilde{w}_t^* = \frac{T \beta_t^{-1}}{\sum_{u=0}^{T-1} \beta_u^{-1}} > 0,$$

which a posteriori verifies that the nonnegativity constraints are inactive at the optimum (the KKT multipliers for the inequalities vanish). Strict convexity then guarantees uniqueness.

Step 6. Degenerate case. If some $\beta_{t_0} = 0$, any feasible allocation supported on $\{t : \beta_t = 0\}$ attains objective value 0, which is minimal. Otherwise, when all $\beta_t > 0$, the interior solution above is the unique minimizer.

Step 7. Optimal value and variance bound. Substituting $\tilde{w}^*$ into the quadratic gives

$$\sum_{t=0}^{T-1} \beta_t (\tilde{w}_t^*)^2 = \sum_{t=0}^{T-1} \beta_t \left(\frac{T \beta_t^{-1}}{\sum_{u=0}^{T-1} \beta_u^{-1}} \right)^2 = \frac{T^2}{\sum_{u=0}^{T-1} \beta_u^{-1}}.$$

Therefore,

$$\text{Var}(g_i^{(\text{rw})}) \leq \mathbb{E}[\hat{A}_i^2] \cdot \frac{T^2}{\sum_{t=0}^{T-1} \beta_t^{-1}}.$$

Step 8. Dual certificate via Cauchy–Schwarz. For any feasible $\{\tilde{w}_t\}$,

$$\left(\sum_{t=0}^{T-1} \tilde{w}_t \right)^2 = \left(\sum_{t=0}^{T-1} \sqrt{\beta_t} \tilde{w}_t \cdot \beta_t^{-1/2} \right)^2 \leq \left(\sum_{t=0}^{T-1} \beta_t (\tilde{w}_t)^2 \right) \left(\sum_{t=0}^{T-1} \beta_t^{-1} \right),$$

which implies

$$\sum_{t=0}^{T-1} \beta_t (\tilde{w}_t)^2 \geq \frac{T^2}{\sum_{t=0}^{T-1} \beta_t^{-1}},$$

with equality if and only if $\tilde{w}_t \propto \beta_t^{-1}$, matching $\tilde{w}_t^*$ and certifying optimality. □

B Training Algorithm: ResT with Rule-Based Rewards

Algorithm 1: Rule-Based Reward Computation

Input: Prediction $(N_P, P_P, \mathbf{v}_P)$; ground truth $(N_G, P_G, \mathbf{v}_G)$ for each tool; format template $\mathcal{T}$; training progress $\bar{v} \in (0, 1)$; weights (β_a, β_f) .

Output: $\mathcal{R}_{i, \text{final}}$.

Format score:

$\mathcal{S}_{i, \text{format}} \leftarrow \mathbf{1}[\text{prediction matches } \mathcal{T} \text{ exactly}]$

Name score:

$r_{\text{name}} \leftarrow \frac{|N_G \cap N_P|}{|N_G \cup N_P|}$

Parameter-name score:

$r_{\text{para}} \leftarrow \sum_{G_j \in G} \frac{|P_G(G_j) \cap P_P(G_j)|}{|P_G(G_j) \cup P_P(G_j)|}$

Parameter-value score:

$r_{\text{value}} \leftarrow \sum_{G_j \in G} \sum_{v \in \mathbf{v}_G(G_j)} \mathbf{1}[P_G(G_j)[v] = P_P(G_j)[v]]$

Normalization:

$Z \leftarrow 1 + |G| + \sum_{G_j \in G} |\mathbf{v}_G(G_j)|$

$\mathcal{S}_{i, \text{acc}} \leftarrow \frac{r_{\text{name}} + r_{\text{para}} + r_{\text{value}}}{Z}$

Dynamic scaling:

$\mathcal{R}_{i, \text{final}} \leftarrow (1 - \bar{v}) (\beta_a \mathcal{S}_{i, \text{acc}} + \beta_f \mathcal{S}_{i, \text{format}})$

return $\mathcal{R}_{i, \text{final}}$

Algorithm 2: ResT: Reshaping Token-level Policy Gradients

Input: Prompt x ; policy π_θ with frozen reference $\pi_{\theta_{\text{old}}}$; group size G ; PPO clip ϵ ; stability constant $\delta > 0$; curriculum proportion $\nu \in [0, 1]$; clipping bounds $(w_{\min}, w_{\max})$; step sizes $(\alpha_f, \alpha_p, \alpha_t)$; entropy-based initializer (Eq. 9).

Output: Updated parameters θ .

Rollout:

for $i = 1$ **to** G **do**

 Sample response $y_i \sim \pi_\theta(\cdot | x)$ containing plain-text and tool-call segments
 Compute reward $\mathcal{R}_{i,\text{final}}$ via Alg. 1

Group-normalized advantage:

$$\mu \leftarrow \frac{1}{G} \sum_{i=1}^G \mathcal{R}_{i,\text{final}}, \quad \sigma \leftarrow \sqrt{\frac{1}{G} \sum_{i=1}^G (\mathcal{R}_{i,\text{final}} - \mu)^2} + \delta$$

for $i = 1$ **to** G **do**

$$\hat{A}_i \leftarrow \frac{\mathcal{R}_{i,\text{final}} - \mu}{\sigma}$$

Token partition and initialization:

// Initialize global token weights (shared across samples)

Let T be a representative sequence length or maximum length

Initialize $\tilde{w}_t$ for all $t = 1, \dots, T$ using entropy-based rule (Eq. 9)

Curriculum reweighting (global schedule):

// Apply curriculum updates once, shared by all sequences

for $t = 1$ **to** T **do**

$$\tilde{w}_t \leftarrow \text{Clip}(\tilde{w}_t, w_{\min}, w_{\max})$$

for $t \in \mathcal{I}_{\text{fmt}}$ **do**

$$\tilde{w}_t \leftarrow \text{Clip}(\tilde{w}_t - \alpha_f \nu, w_{\min}, w_{\max})$$

for $t \in \mathcal{I}_{\text{name}}$ **do**

$$\tilde{w}_t \leftarrow w_{\max}$$

for $t \in \mathcal{I}_{\text{para}}$ **do**

$$\tilde{w}_t \leftarrow \text{Clip}(\tilde{w}_t + \alpha_p \nu, w_{\min}, w_{\max})$$

for $t \in \mathcal{I}_{\text{thk}}$ **do**

$$\tilde{w}_t \leftarrow \text{Clip}(\tilde{w}_t + \alpha_t \nu, w_{\min}, w_{\max})$$

// Normalize globally

$$\bar{w} \leftarrow \frac{1}{T} \sum_{t=1}^T \tilde{w}_t$$

for $t = 1$ **to** T **do**

$$\omega_t \leftarrow \frac{\tilde{w}_t}{\bar{w} + \delta}$$

PPO-style token-level objective and update:

for $i = 1$ **to** G **do**

for $t = 1$ **to** T^i **do**

$$r_{i,t} \leftarrow \frac{\pi_\theta(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})}$$

$$g_{i,t} \leftarrow \min(r_{i,t} \hat{A}_i, \text{clip}(r_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i)$$

$$\mathcal{L}_{\text{ResT}}(\theta) \leftarrow -\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^T -\frac{\omega_t}{T} \cdot g_{i,t}$$

Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{ResT}}(\theta)$

Helper: $\text{Clip}(u, a, b) = \min(\max(u, a), b)$. Sets $\mathcal{I}_{\text{fmt}}, \mathcal{I}_{\text{name}}, \mathcal{I}_{\text{para}}, \mathcal{I}_{\text{thk}}$ are obtained via deterministic token tagging of the generated sequence. Eq. 9 provides the entropy-based initializer for $\tilde{w}_t$.

C Complete Experiments Results

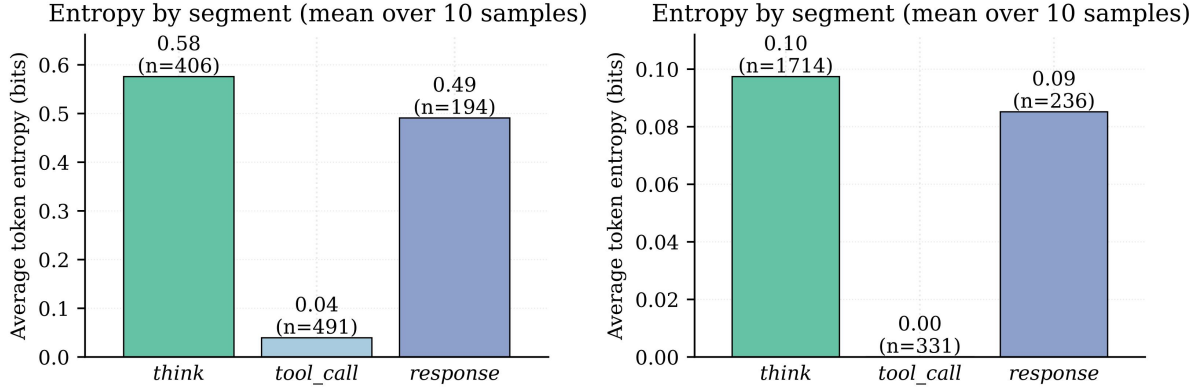


Figure 4: Average region entropy on Llama-3.2-3B-Instruct (left) and Qwen3-1.7B (right). For each model, we randomly sample 10 instances and compute the entropy distribution over generated tokens. Tokens are partitioned into functional regions—reasoning, tool invocation, and final response.

Table 5: **API-Bank Test Results on Llama3.2-3B-Instruct model.** We report the highest accuracy achieved for each method after hyper-parameter tuning, with a breakdown by difficulty level.

Model	Overall Acc	Level 1 Acc	Level 2 Acc	Level 3 Acc
Llama-3.2-3B-Instruct(Base)	37.35%	43.61%	32.84%	20.61%
Llama-3.2-3B-Instruct(SFT)	46.4%	52.88%	29.85%	35.11%
Llama-3.2-3B-Instruct(TSFT)	49.25%	57.14%	31.34%	34.35%
Llama-3.2-3B-Instruct(RSFT)	51.26%	57.89%	29.85%	41.98%
Llama-3.2-3B-Instruct(GRPO)	59.13%	65.66%	52.24%	42.75%
Llama-3.2-3B-Instruct(SFT+GRPO)	52.76%	61.15%	43.28%	32.06%
Llama-3.2-3B-Instruct(Ours, ResT)	62.81%	69.42%	53.73%	47.33%

Table 6: **BFCL Benchmark Results on Single Turn Scenarios (last updated June 14, 2025).** All metrics are calculated using the official scripts.

Model	Non-Live AST Acc	Live Acc	Relevance Detection	Irrelevance Detection
Qwen3-1.7B (Base)	83.27%	72.95%	83.33%	85.49%
Qwen3-1.7B (SFT)	82.90%	74.06%	72.22%	78.02%
Qwen3-1.7B (TSFT)	83.67%	74.59%	77.78%	78.04%
Qwen3-1.7B (RSFT)	82.46%	74.19%	66.67%	76.53%
Qwen3-1.7B (GRPO)	83.00%	72.95%	72.22%	73.42%
Qwen3-1.7B (SFT+GRPO)	85.31%	72.10%	66.67%	76.94%
Qwen3-1.7B (Ours, ResT)	84.65%	73.43%	66.67%	76.30%
Qwen3-4B-2507 (Base)	87.98%	79.52%	77.78%	79.18%
Qwen3-4B-2507 (SFT)	89.04%	80.94%	83.33%	81.90%
Qwen3-4B-2507 (TSFT)	87.31%	80.63%	83.33%	81.39%
Qwen3-4B-2507 (RSFT)	87.69%	80.59%	88.89%	82.02%
Qwen3-4B-2507 (GRPO)	89.40%	79.08%	83.33%	81.52%
Qwen3-4B-2507 (SFT+GRPO)	89.46%	80.36%	77.78%	80.22%
Qwen3-4B-2507 (Ours, ResT)	88.65%	81.12%	77.78%	79.67%
Qwen3-8B (Base)	88.81%	78.54%	77.78%	79.08%
Qwen3-8B (SFT)	82.58%	73.70%	72.22%	75.62%
Qwen3-8B (TSFT)	81.88%	75.26%	83.33%	76.72%
Qwen3-8B (RSFT)	81.08%	73.83%	77.78%	77.38%
Qwen3-8B (GRPO)	88.50%	80.85%	83.33%	82.24%
Qwen3-8B (SFT+GRPO)	85.90%	75.70%	77.78%	75.96%
Qwen3-8B (Ours, ResT)	90.08%	79.03%	94.44%	77.62%
Qwen3-14B (Base)	88.17%	79.96%	72.22%	81.01%
Qwen3-14B (SFT)	77.31%	71.83%	77.78%	69.95%
Qwen3-14B (TSFT)	79.90%	71.97%	83.33%	72.48%
Qwen3-14B (RSFT)	71.81%	73.39%	55.56%	85.44%
Qwen3-14B (GRPO)	89.10%	79.74%	77.78%	79.31%
Qwen3-14B (SFT+GRPO)	86.56%	76.59%	77.78%	75.41%
Qwen3-14B (Ours, ResT)	89.10%	80.81%	77.78%	80.56%

D Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization (GRPO) serves as a critic-free, on-policy optimization algorithm designed for Large Language Models (LLMs). It presents an efficient alternative to Proximal Policy Optimization (PPO) by constructing a relative baseline for advantage estimation. This approach circumvents the need to learn a separate value network, thereby avoiding dependencies on methods like Generalized Advantage Estimation (GAE) which require a value function.

The core idea of GRPO is to dynamically construct a baseline for each sampled output using the reward distribution of a set of candidate outputs generated from the same input prompt. Specifically, for a given prompt q , we first sample a set of S complete output sequences $\mathcal{O} = \{o_1, o_2, \dots, o_S\}$ from the current policy $\pi_{\theta_{old}}$. A scalar reward $r_i = R(q, o_i)$ is then computed for each output o_i .

The empirical mean of these rewards serves as the baseline for evaluating each output, i.e., $b(q) = \text{mean}(\{r_1, \dots, r_S\})$. The advantage for each rollout o_i is then normalized using the standard deviation of the rewards to obtain the normalized advantage estimate $\hat{A}_i$:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_S\})}{\text{std}(\{r_1, \dots, r_S\}) + \delta} \quad (20)$$

where δ is a small constant added for numerical stability to prevent division by zero. This "intra-group relative" advantage estimation effectively reduces reward variance.

During the policy update phase, this sequence-level advantage value $\hat{A}_i$ is applied to every timestep (token) within that sequence. GRPO performs policy updates by maximizing a clipped surrogate objective, analogous to PPO:

$$\begin{aligned} \mathcal{L}^{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, o \sim \pi_{\theta_{old}}} & \left[\sum_{t=1}^{|o|} \min\left(\rho_t(\theta) \hat{A}, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}\right) \right. \\ & \left. - \beta \text{KL}(\pi_{\theta}(\cdot|q) \parallel \pi_{\text{ref}}(\cdot|q)) \right]. \end{aligned} \quad (21)$$

where $\rho_t(\theta) = \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}$ is the likelihood ratio of the new and old policies at timestep t . $\hat{A}$ corresponds to the normalized advantage of the current sequence o . The hyperparameters ϵ and β control the clipping range and the strength of the KL-divergence penalty against a reference policy π_{ref} , respectively.

By adopting this formulation, GRPO retains the stability of PPO's trust-region updates while eliminating the complexities of training a value network, offering a simple yet effective framework for fine-tuning LLMs with reinforcement learning.